



Introduction to Data Science

Assignment 5

Instructors: Dr. Yaghoobzadeh, Dr. Razavi **TA(s):** Soroush Esfahanian, Erfan Falahati, Farshad Hessami

Deadline: 28th of Tir

Introduction



This assignment builds practical data-science skills through four distinct tasks using real-world datasets. You will engineer features and apply semi-supervised learning (pseudo-labeling, active learning) to predict Formula1 pit-stop strategies under label scarcity, fine-tune a Persian transformer to recognise semantic equivalence across formal and informal registers, prompt a large language model via LangChain to simulate naïve human responses as a test of machine theory-of-mind, and segment football players in images using clustering techniques without ground-truth masks. Each task requires careful data preparation, rigorous evaluation, and critical interpretation of results, mirroring the end-to-end workflow expected in professional analytics environments.

Task 1 – F1 Pit Strategy via SSL & Active Learning

Formula 1 (F1) is the highest class of international single seater auto racing. Each season consists of a series of races called Grands Prix held at circuits around the world. F1 victory is decided not only by driver skill, but by engineering, aerodynamics, and critical race strategy. One of the most consequential strategic decisions a team makes during a race is the pit stop.

A pit stop is when a car leaves the track, enters the pit lane, and pauses briefly so mechanics can change tires, make repairs, or adjust the car. Each stop costs roughly 20-30 seconds of race time.

There are four main pit stop strategies teams use:

- **Undercut:** The car pits slightly before the rival ahead, gets fresh tyres, laps faster, and rejoins in front once the rival pits. Requires closing the gap to the car ahead.
- **Overcut:** A strategic move where a driver remains on track longer than an immediate rival who has already pitted. By staying out, the driver exploits "clean air" to set fast lap times on older tyres, capitalizing on the rival's struggle to warm up fresh tyres or their entrapment in traffic. This allows the driver to either overtake a rival ahead or defend their position against a rival behind once they finally pit.
- **Standard:** A planned stop based purely on tyre life targets with no tactical intent to overtake or defend. The most common strategy
- **Emergency:** An unplanned or reactive stop triggered by tyre damage, a safety car, heavy rain, or sudden tyre failure. High urgency, no prior planning.

This task asks you to predict which of these four strategies was used for a given pit stop, using only the race and lap features available at the moment of the decision and to do so with very limited labeled data. You will engineer features from multiple raw CSV files, build supervised baselines, and then apply two semi-supervised learning techniques: Pseudo Labeling and Active Learning.

Dataset Description

You will use the [Formula 1 World Championship \(1950-2024\)](#) dataset from Kaggle, compiled from the Ergast Developer API.

Files you may need: races.csv, pit_stops.csv, lap_times.csv, results.csv, driver_standings.csv, constructor_standings.csv and constructors.csv.

After downloading and joining the files you will build a lap level feature matrix for all 2014–2024 pit stops, assign strategy labels, and then produce two CSV files via a random split:

- **labeled.csv:** A randomly sampled ~10% of all 2014–2024 pit stops with all engineered features AND the strategy label. This simulates the scarce human annotated data available.
- **unlabeled.csv:** The remaining ~90% of 2014–2024 pit stops with all engineered features but NO labels. Both files come from the same era and the same distribution.

1. Data Loading & Feature Engineering (25 Points)

Raw CSV files from the Ergast dataset are relational. Each file represents one table linked by integer IDs. Your first job is to join them and build a rich, lap level feature matrix where each row is one pit stop event.

1.1 Download and Inspect (3 pts)

Download and load CSVs with pandas, display shapes/head rows, and document missing values, data types, and key IDs.

1.2 Join the Tables (5 pts)

Filter to 2014–2024 seasons first, then merge the CSV files into a single DataFrame. Include at minimum:

- Pit stop lap number and stop duration in milliseconds
- Driver's grid position at race start and current race position on the pit lap
- Constructor (team) name, circuit name, and race year
- Driver and constructor championship points at the time of the stop

Hint: To get championship points before the race, join with the previous raceId for each driver/constructor. You may use the round column and year ordering to identify the prior race.

1.3 Engineer Lap Level Features (17 pts)

Part A: Required Features (7 pts)

For each pit stop, compute the following features from the provided dataset:

- **lap_time_delta (ms):** Driver's lap time on the pit lap minus their personal best in that race. A large positive value signals tyre degradation.
- **laps_since_last_pit:** Laps elapsed since the driver last pitted. Proxy for tyre age.
- **laps_remaining:** Laps remaining in the race after this stop.
- **approx_gap_ahead (s):** Gap in seconds to the car immediately ahead (Guideline: Calculate the cumulative race time for each driver up to the pit lap from lap_times, then subtract the cumulative time of the driver in the immediately higher position).
- **approx_gap_behind (s):** Gap in seconds to the car immediately behind (approximate from positions and lap times using a corresponding cumulative time difference).
- **safety_car_likely (0/1):** Flag 1 if multiple drivers on the same lap are unusually slow (safety car proxy).

Part B: Open Ended Feature Engineering (10 pts Bonus)

Engineer at least 2 additional features that you believe capture tyre degradation, competitive pressure, or strategic context. You must justify each feature with an F1 domain argument and show its discriminative power with a visualization

2. Strategy Labeling & Dataset Split (5 Points)

2.1 Define Labeling Rules (3 pts)

Instead of using predefined labels, you must design and implement your own rule based heuristics to assign one of four strategy labels to each 2014–2024 pit stop: Standard, Undercut, Overcut, or Emergency.

Use the features engineered in Section 1.3 (or develop your own customized features) to formulate these conditions. To ensure your rules reflect real world Formula 1 racing dynamics, your final annotated dataset should roughly approximate the following logical class distributions:

Standard (~65% – 75%): The baseline scenario. The vast majority of pit stops are scheduled, routine tyre changes.

Undercut (~12% – 18%): Tactical stops triggered when a driver is close to the car ahead, leveraging fresh tyres to pass them during the pit cycle.

Emergency (~8% – 12%): Highly disruptive, unplanned stops. This includes pitting under safety car/VSC conditions, dealing with sudden punctures, or managing extreme unexpected tyre degradation.

Overcut (~4% – 8%): Strategic extension of a stint to gain an advantage. This is generally less common in the modern Pirelli era and highly track dependent.

2.2 Random Split into labeled.csv and unlabeled.csv (2 pts)

1. Apply `train_test_split` with `test_size=0.90` to split all labeled pit stops into a ~10% labeled set and a ~90% unlabeled pool and `stratify=y` (where `y` is the strategy label column) to ensure proportional class distribution in both files.

2. Save the ~10% portion (with strategy column) as `labeled.csv`.

3. Save the ~90% portion (strategy column dropped) as `unlabeled.csv`.

4. Verify: print class counts in `labeled.csv`, shapes of both files, and 3 sample rows from each.

5. Confirm the split is fair: compare class proportions in `labeled.csv` vs the full pool. They should be similar because of stratification.

3. Exploratory Data Analysis (12 Points)

3.1 Class Distribution (3 pts)

- Plot a bar chart of strategy class counts in `labeled.csv` with percentage labels.
- Plot a side by side bar chart comparing class proportions in `labeled.csv` vs the full 2014–2024 pool. Confirm the stratified split preserves the distribution.

3.2 Feature Distributions by Strategy (4 pts)

- For each of the four key features (`laps_since_last_pit`, `stop_duration_ms`, `lap_time_delta`, `laps_remaining`), plot overlapping histograms coloured by strategy class.
- Which features most clearly separate classes? Explain the F1 reasoning behind each.
- Plot a correlation heatmap of all numeric features.

3.3 Split Quality Check (2 pts)

Since labeled and unlabeled data come from the same era and distribution, the feature distributions should be statistically similar.

- For at least 4 features, plot side by side histograms comparing labeled vs unlabeled.
- Run a Kolmogorov-Smirnov (KS) test for each feature. You should observe $p > 0.05$ (no significant difference). Explain why this matters.

(Note: The KS test is designed for continuous variables; it is not suitable for binary features such as `safety_car_likely`).

3.4 Constructor & Circuit Patterns (3 pts)

- Which constructors use the most Undercut strategies? Plot a grouped bar chart.
- Are Emergency stops more common at specific circuits? Visualise with a heatmap.
- At which circuits is the `approx_gap_ahead` smallest on average? Why might this be?

4. Supervised Learning Baseline (15 Points)

Before using semi-supervised methods, establish how well a model trained only on `labeled.csv` performs. This baseline is your reference point for measuring SSL improvement.

4.1 Preprocessing (3 pts)

Drop `safety_car_likely` and `stop_duration_ms` to prevent rule reverse engineering. Encode categorical features (`constructor_name`, `circuit_name`) with justification, scale numeric data via `StandardScaler`, stratify split `labeled.csv` into train/validation/test (80/10/10%), and address class imbalance using `class_weight='balanced'` (explain its necessity).

4.2 Train Multiple Classifiers (6 pts)

Train and compare at least three classifiers: Logistic Regression, Random Forest Classifier and Gradient Boosting Classifier.

To prevent complete rule memorization and ensure a smooth learning curve for the subsequent SSL and Active Learning tasks, apply regularization to the tree-based models (e.g., limit `max_depth` or increase `min_samples_leaf`).

Use the validation Macro F1-score to select the best model. Report all three in a table.

4.3 Evaluate on Test Set (6 pts)

For your best classifier, compute and report:

- Accuracy, Macro Precision, Macro Recall, Macro F1-score
- Per class F1-score and confusion matrix heatmap
- Feature importance plot. Which features drive predictions most?

5. Semi-Supervised Learning: Pseudo Labeling (20 Points)

Pseudo labeling is an iterative SSL technique where a model trained on labeled data predicts labels for unlabeled data. High-confidence predictions (above a threshold) are added to the training pool as pseudo-labels, and the model is iteratively retrained. Key concepts:

- **Confirmation Bias:** If the model makes systematic errors, those wrong labels get added and reinforced over rounds. Careful threshold selection is critical.
- **Confidence Threshold:** Controls the quality vs. quantity trade-off. Too low → noisy additions. Too high → too few samples added.

5.1 Implementation (13 pts)

1. Train the best baseline classifier on the training portion of labeled.csv (the %80 split from section 4.1)
2. Use `predict_proba()` on the unlabeled pool. Confidence = max probability across all classes.
3. Select samples where confidence $\geq$ threshold (start with 0.75) and assign the argmax class as a pseudo label.
4. Combine the original labeled training data with the newly pseudo-labeled samples.
5. Retrain the model on the expanded set and evaluate on the held-out test set.
6. Repeat steps 2–5 for at least 5 rounds or until fewer than 50 new samples are added.
7. Also run the full experiment with thresholds 0.60 and 0.85. Compare all three.

5.2 Analysis (7 pts)

- Plot how many pseudo labels were added per round, broken down by class. Does the model confidently pseudo-label certain strategies more than others?
- Plot the test Macro F1-score after each round for all three threshold values on the same figure.

- At which round and threshold do you observe the best improvement? Where does it plateau?
- Set threshold to 0.55. What happens to the learning curve? Explain in terms of confirmation bias.

6. Active Learning (25 Points)

Active Learning interactively queries an expert to label the most informative samples, mimicking an F1 analyst reviewing limited ambiguous pit stops. Three query strategies identify uncertainty: Least Confidence selects the lowest predicted top class probability; Margin targets the smallest difference between the top two probabilities; and Entropy chooses the highest Shannon entropy across all classes.

6.1 Implementation (15 pts)

1. Start with the initial small labeled training set and train your best baseline classifier.
2. For each active learning round, compute uncertainty scores for all samples in the unlabeled pool using your chosen query strategy.
3. Select the top k most uncertain samples (use $k = 20$ per round).
4. Simulate human annotation: use the same rule-based labeling function from section 2 to assign labels to the selected k samples. This simulates querying a domain expert.
5. Add the newly labeled samples to the training set and retrain the model.
6. Record the model's test set Macro F1-score after each round.
7. Run for at least 8 rounds. Implement and compare all three query strategies.

6.2 Analysis (10 pts)

- Plot model performance (test Macro F1) vs. total labeled samples for all three query strategies on one figure. Add a horizontal baseline line.
- Which strategy reaches baseline performance fastest? Report the exact round and label count where each first matches the baseline.
- Inspect which classes were queried most in round 1. Are Emergency stops overrepresented? Why does this make sense?

- Compare Active Learning vs. Pseudo Labeling: AL uses $k=20$ human labels per round; PL uses up to 800 model generated labels per round. Which approach reaches the same F1 with fewer total labeled samples?
- Discuss the trade-off between annotation cost (manual labeling) and data volume (pseudo labels). When would each method be preferred in a real F1 team scenario?

7. Comparative Analysis (8 Points)

7.1 Summary Table (2 pts)

Create a summary table:

- Rows: Baseline | Pseudo Labeling Round 1 ... Final | Active Learning (each strategy, final round)
- Columns: Total Labeled Samples, Accuracy, Macro Precision, Macro Recall, Macro F1

7.2 ROC Curves & AUC (4 pts)

- For the Baseline, best Pseudo Labeling model, and best Active Learning model, plot one vs. the rest ROC curves for all four strategy classes on a 1×3 subplot grid.
- Report per class AUC and mean AUC for each method.
- Which class has the lowest AUC across all methods? Relate this to its label count.

7.3 Learning Curves (2 pts)

- Plot Macro F1 vs. total labeled samples on one figure showing Pseudo Labeling (best threshold), all three AL strategies, and the Baseline line.
- At what labeled set size does each method stop providing meaningful gains?

Final Discussion

Wrap up by thinking through these final questions. Make sure you understand them, but you don't need to put the answers in your report.

1. Which method Pseudo Labeling or Active Learning performed better, and why? Consider the effect of label scarcity and class imbalance.

2. In this task, labeled and unlabeled data come from the same distribution. How do you expect results would change if the unlabeled data came from a different era with different tyre rules and scoring systems?

3. In a real F1 team with a live data pipeline, which method would you deploy, and what safeguards would you put in place to avoid confirmation bias?

Task 2 – Semantic Equivalence in Formal and Informal Persian

Natural language allows the same core meaning to be expressed in a wide variety of structural forms. This phenomenon is also observed in the Persian language, where informal conversational speech exhibits profound phonological, lexical, and syntactic deviations from its formal written counterpart. While humans effortlessly bridge these stylistic variations to recognize semantic equivalence, enabling computational models to capture abstract meaning beyond surface-level token alignment remains a hurdle in Persian Natural Language Processing (NLP).

In this assignment, you will examine the computational mechanics of semantic representation. Moving from traditional distributed word representations to state-of-the-art context-aware transformers, you will build an end-to-end data-centric pipeline to identify semantic alignment across stylistic boundaries.

Dataset Description

This project utilizes the [ParsMap](#) dataset, a curated corpus containing pairs of semantically equivalent formal and informal Persian sentences. The dataset is structured to facilitate an empirical comparison between surface lexical adjustments and core semantic consistency. It contains two columns, namely `formalForm` and `informalForm`, each holding a string. You will use this dataset across all phases to analyze linguistic patterns, measure embedding similarities, and train a supervised transformer model.

1. Exploratory Data Analysis and Linguistic Properties (29 Points)

Raw textual data must be comprehensively understood before feeding it into any modeling architecture. In this section, you will dissect the structural and vocabulary discrepancies inherent to formal and informal Persian.

1.1 Dataset Exploration & Transformation Mapping (8 Points)

Before executing structural counts, conduct a qualitative review of stylistic shift dynamics within the dataset.

- Load the dataset into a pandas DataFrame and sample a collection of random sentence pairs.
- Display both the formal and informal variants side by side.
- Identify and document specific linguistic modifications occurring during the formal-to-informal shift.

1.2 Sentence Length Distribution Analysis (7 Points)

Quantify structural differences by measuring sequence lengths across both domains.

- For every sentence in the corpus, calculate two distinct metrics: Word Count (via whitespace splitting) and Character Count.
- Calculate the statistical metrics for word count and character count (mean, median, standard deviation, etc.) for formal and informal sentences, and compare the results. Justify any differences in the values.
- (Optional) Generate comparative visualizations contrasting the statistical distributions for both formal and informal subsets. You can use overlaying Histograms, Kernel Density Estimate (KDE) plots, and Boxplots to isolate outliers.

1.3 Stylistic Vocabulary Analysis (7 Points)

Examine the overlap and uniqueness of lexicons between the two textual registers.

- Construct two separate vocabularies representing the formal and informal corpora.
- Compute total vocabulary sizes, isolate words unique to each register, and extract the top 20 most frequent terms for each category.
- Compare the vocabulary size of formal and informal corpora and justify the differences. Which corpora has a larger vocabulary? Why?

1.4 Transformer Tokenization Complexity (7 Points)

Modern subword tokenization schemes process sequences differently than classical whitespace splits, especially when dealing with non-standard dialects or casual scripts.

- Load the native tokenizer associated with the pre-trained ParsBERT architecture (HooshvareLab/bert-fa-base-uncased).
- Tokenize the data once using whitespace splitting and once with the ParsBert tokenizer.

- Compare subword fragmentation against standard whitespace tokenization across a number of selected sentence pairs. Spot the differences.

Token Inflation Factor (TIF): To mathematically isolate the tokenization overhead introduced by casual language variants, calculate the ratio:

$$\text{TIF} = \frac{\text{Number of Transformer Tokens}}{\text{Number of Whitespace Tokens}}$$

- Compute the TIF for every sentence in the dataset. Plot and compare the TIF distributions for both the formal and informal pools. Which one has a higher TIF? Why?

2. Representing Meaning with Semantic Embeddings (38 Points)

Vector spaces allow us to map text into continuous representations where distance correlates with semantic similarity. In this section, you will evaluate whether static and contextual embeddings can recognize that formal and informal expressions carry identical semantics.

2.1 Static Word Representations via Word2Vec (10 Points)

- Research and understand the Word2Vec model, including its underlying principles, training objectives, and the differences between the Continuous Bag-of-Words (CBOW) and Skip-Gram architectures. Understand how Word2Vec learns distributed word representations and captures semantic relationships between words.
- Utilize the `gensim.models.Word2Vec` library to train a continuous skip-gram or CBOW model over the entire integrated ParsMap text corpus.
- Select an explicit set of target vocabulary terms (preferably among the more frequent words) containing both formal keywords and their informal equivalents. Extract their top 5 nearest neighbors using Cosine Similarity.
- Examine whether formal and informal expressions converge into shared semantic spaces.

2.2 Sentence Similarity & Evaluation Framework (11 Points)

- Construct a sentence-level embedding function by computing the element-wise average of constituent Word2Vec token vectors for every sentence.
- Formulate two diagnostic evaluation groups:
 - **Group A (Positive Controls):** True aligned formal-informal pairs extracted from the dataset.
 - **Group B (Negative Controls):** Corrupted pairs created by matching formal sentences with completely random informal sentences.
- Compute the Cosine Similarity score for every pair across both groups. Plot the resulting distributions for Group A and Group B on a single chart.
- Find some equivalent pairs that have a Cosine Similarity score less than 0.4 and print them out. Can you justify why Word2Vec has assigned a low similarity score to them?

2.3 Contextual Representations with ParsBERT (11 Points)

- Initialize a context-aware transformer by loading the pre-trained ParsBERT model (HooshvareLab/bert-fa-base-uncased) via the Hugging Face transformers library.
- Extract dense sentence representations. You may use the pooled output ([CLS] token embedding) or apply mean pooling across the final hidden state sequence layer.
- Re-run the exact similarity experiment outlined in Task 2.2 using these contextual vectors. Compute similarity scores for Group A and Group B, and plot their distributions.
- Compare the similarity distributions of ParsBERT and Word2Vec. Justify any differences.

2.4 High-Dimensional Space Visualization (6 Points)

- Reduce high-dimensional sequence embeddings into a two-dimensional plane using Principal Component Analysis (PCA) alongside non-linear reduction techniques (t-SNE or UMAP).
- Generate separate 2D scatter plots for the Word2Vec and ParsBERT embedding spaces. Use different colors to distinguish formal and informal sentence representations. For 30 randomly selected formal-informal sentence pairs, draw straight lines connecting the corresponding points to visualize the distance between equivalent expressions in each embedding space.

- Compare and interpret the two visualizations. Discuss the difference between the two plots and the overall clustering patterns. How do they correspond to the dynamics of Word2Vec and ParsBERT?
- Based on the plots, determine which model (Word2Vec or ParsBERT) better aligns semantically equivalent formal and informal expressions. Justify your answer.

3. Supervised Semantic Equivalence Classification (43 Points)

While raw embedding distance offers an unsupervised metric for semantic proximity, fine-tuning a transformer model allows its multi-head attention layers to learn the explicit mapping of stylistic shifts. In this section, you will optimize ParsBERT for a sequence-pair classification task.

3.1 Binary Classification Dataset Construction (9 Points)

Convert the parallel corpus into a supervised learning framework.

- **Positive Class (Label 1):** Aligned formal-informal pairs from the original dataset (True Semantic Equivalents).
- **Negative Class (Label 0):** Unaligned pairs generated by matching formal sentences with non-equivalent, random informal entries. Maintain a strict 1:1 class balance.
- **Data Leakage Precaution:** To avoid data leakage, ensure that no individual sentence appears in more than one split (train/val/test) in any role (positive or negative pair). Shuffle the sentences and assign them to splits *before* generating negative pairs.
- Split the generated dataset into separate Training (80%), Validation (10%), and Testing (10%) partitions. Ensure no cross-contamination or data leakage occurs between splits.

3.2 Transformer Input Encoding (8 Points)

Transformers require highly structured inputs to process sentence pairs simultaneously.

- Pass pairs through the ParsBERT tokenizer to generate proper model inputs.
- Extract and display a single tokenized sample. Document and explain the operational role of the following arrays:
 - `input_ids` (including the placement of special tokens like [CLS] and [SEP]).

- attention_mask (handling padding variations across batches).
- token_type_ids / segment_ids (distinguishing sequence A from sequence B).

3.3 Fine-Tuning ParsBERT (16 Points)

Fine-tune the architecture to resolve semantic equivalence under style variations.

- Append a binary classification head on top of the pre-trained ParsBERT model (AutoModelForSequenceClassification).
- Configure optimization hyperparameters: set an appropriate learning rate (e.g., 2×10^{-5} to 5×10^{-5}), employ an AdamW optimizer, utilize a linear scheduler with warmup, and select a manageable batch size.
- Train the architecture across multiple epochs using available hardware (Kaggle, Google Colab, or local GPU resources). Track training loss and validation loss at regular step intervals.
- Plot training and validation loss curves to monitor convergence and guard against overfitting.

3.4 Evaluation, Success Analysis, and Live Inference (10 Points)

Evaluate the model on the test partition, examine its successes and failures, and implement an interactive evaluation tool.

- **Quantitative Evaluation:** Execute inference over the held-out test split. Compute comprehensive classification performance metrics: Accuracy, Precision, Recall, and F1-Score (macro-averaged).
- **Success Analysis:** Isolate and investigate several correctly classified samples from the test set. Examine both *True Positives* (where equivalent pairs were accurately matched) and *True Negatives* (where random pairings were accurately rejected). Analyze the semantic stability or vocabulary cues that facilitated these successful predictions despite register shifts.
- **Error Analysis:** Isolate several incorrectly classified examples (*False Positives* and *False Negatives*). Display the text pairs alongside their true labels and predicted probabilities, discussing possible reasons for model confusion.
- **Live Inference Function:** Develop a standalone, interactive Python function:

```
def predict_equivalence(formal_sentence: str, informal_sentence: str) -> dict:
    """
    Receives a formal and an informal Persian sentence string,
    tokenizes them as a sentence pair, performs inference using
    the fine-tuned ParsBERT model, and returns the equivalence
    decision along with the confidence score.
    """
    pass
```

Demonstrate the utility of this function by testing it on novel, out-of-dataset sentence pairs that you craft yourself.

Final Discussion

As a final discussion and conclusion, reflect on the following questions and ensure you can answer them, but there's no need to include the answers in your report.

1. What are the primary structural, morphological, and syntactic differences between formal and informal Persian discovered during your exploration?
2. Why does the averaging of static word embeddings (Word2Vec) encounter structural limitations when mapping sentences across different stylistic domains?
3. What specific architectural components enable pre-trained transformers (ParsBERT) to successfully capture semantic alignment despite extensive lexical changes?
4. How does the process of supervised fine-tuning modify the model's inner mechanics compared to utilizing its frozen, pre-trained representations? Investigate the differences between the tokenization approach utilized in sections 2 and 3, if any.

Task 3 – Simulating Human Knowledge Gaps: LLMs, Theory of Mind, and Science QA

Large Language Models are increasingly used not just to answer questions, but to *simulate other agents*, students, customers or naive users for purposes like tutoring system design, dataset augmentation, and behavioral research. Doing this well requires a form of Theory of Mind (ToM): the model must reason about what *another agent* believes or doesn't believe, and predict that agent's behavior accordingly, not simply produce the correct answer itself.

In this task, you will use an LLM (via LangChain) to (1) decompose science questions into their underlying prerequisite concepts, and (2) generate simulated answers from the perspective of a person who lacks knowledge of some of those concepts. You will then analyze where and how the model's simulations succeed or fail, including cases where the model "leaks" knowledge it was told to suppress.

Dataset: OpenBookQA: a multiple-choice science question dataset (originally from Allen AI), where each question is paired with a gold (correct) answer and three distractor options, designed to require both core scientific facts and broader common-sense knowledge to answer correctly. The dataset is available via the Hugging Face datasets library (`load_dataset("allenai/openbookqa", "main")`) or as raw JSONL files from the [OpenBookQA GitHub repo](#). Download and load it yourself; document its structure (fields, number of questions, format of choices/answers, train/validation/test splits) before proceeding.

1. LLM & LangChain Fundamentals (22 Points)

1.0 API Setup

For this entire assignment, you will use Google AI Studio's free tier API (<https://aistudio.google.com>). This gives free, rate-limited access to Google's models via an API key.

Model selection: use this priority order, falling back only if you hit rate limits or quota errors with the higher option:

1. Gemma 4 31B
2. Gemma 4 26B

3. Gemini 3.1 Flash-Lite

Document in your notebook which model(s) you ended up using and why (e.g., "Used Gemma 4 31B for Sections 1–3, switched to Gemini 3.1 Flash-Lite for Section 4 due to rate limiting"). If you hit rate limits frequently: you are allowed to create multiple Google accounts and obtain multiple API keys, then rotate between them (e.g., a simple wrapper that switches keys when one hits a rate-limit error). This is optional but recommended if you find generation steps later in the assignment slow. Document if you do this.

1.1 Setup & Basic Calls (4 pts)

1. Obtain an API key from Google AI Studio. Set it as an environment variable (e.g., `GOOGLE_API_KEY`).
2. Configure LangChain to use your selected model (start with Gemma 4 31B) via `langchain-google-genai`.
3. Send a basic prompt and display the raw response.
4. Briefly document: which model(s) you used, any rate limit/quota errors you encountered, and how you handled them (including whether you used multiple API keys).

1.2 Prompt Templates (5 pts)

1. Define a PromptTemplate for a simple question-answering task (input: question + choices, output: free-text answer).
2. Run this template on 3 sample questions from your loaded dataset and display the outputs.

1.3 Structured Output Parsing (5 pts)

1. Define a Pydantic schema for a structured response containing at least: answer (the selected choice), reasoning (free text), and confidence (numeric, 0–1).
2. Use a structured output parser (e.g., `PydanticOutputParser` or `with_structured_output`) to get responses in this format for the same 3 questions.

3. Trigger and document at least one case where the model's output failed to parse correctly. Show how you handled it (e.g., retry, OutputFixingParser, or a manual repair step).

1.4 Dataset Enrichment: Adding Subject & Difficulty Labels (8 pts)

OpenBookQA does not include subject area or difficulty annotations. In this section, you'll use the LLM and structured output parsing to generate these labels yourselves - a common real-world use of LLMs for dataset annotation/enrichment. The 100 questions you select here will be the working dataset for the rest of this assignment. all later sections (including Section 3's question selection) draw from this set, not the full OpenBookQA dataset.

1. Randomly sample 100 questions from OpenBookQA (set a random seed and document it for reproducibility).
2. Define a Pydantic schema for an annotation response containing at least:
 - subject_area:** one of a fixed set of categories you define (e.g., physics, biology, chemistry, earth_science, astronomy, other). pick a small, fixed list of 4-6 categories *before* running this, so labels are consistent across questions.
 - difficulty:** an integer 1-5 (1 = basic recall, 5 = requires multi-step reasoning or combining multiple concepts), with a short rubric you define for what each level means.
 - reasoning** (brief justification for the labels. useful for spot-checking).
3. For each of the 100 questions, prompt the LLM (using your structured output parser from 1.3) with the question text and choices, and collect subject_area, difficulty, and reasoning.
4. Save the enriched dataset as enriched_questions.csv (original fields + subject_area, difficulty, reasoning).
5. Spot-check 10 random rows: do you agree with the assigned subject_area and difficulty? Report your agreement rate with brief notes on disagreements.
6. Plot the distribution of subject_area and difficulty across your 100 questions (two simple bar charts).

2. Theory of Mind Background (17 Points)

2.1 What is Theory of Mind? (4 pts)

In a few sentences, explain what Theory of Mind (ToM) is, including the distinction between first-order ToM ("what does person A believe?") and second-order ToM ("what does A believe B believes?"). Briefly mention the false-belief task paradigm (e.g., the Sally-Anne test) as a classic way of testing ToM.

2.2 Hands-On: Testing a Classic False-Belief Task (7 pts)

1. Write out 2-3 classic false-belief scenarios as prompts (e.g., a Sally-Anne-style scenario, or adapt one from a benchmark like ToMi). Use your structured output parser from Section 1.3 to get the model's answer + reasoning.
2. Does your model pass these? Show the actual outputs.
3. If it passes, does the reasoning suggest genuine perspective-tracking, or could it be pattern-matching on a familiar template? Try a slightly modified/novel version of one scenario and see if performance holds; report what you find.

2.3 Connecting ToM to This Task (6 pts)

In 2-3 paragraphs, explain:

- Why generating "what would a person who doesn't know X answer" is a ToM-relevant task, distinct from simply answering the question correctly.
- What it would mean for the model to "fail" at this task? distinguish at least two different failure modes (think about what could go wrong, even if you haven't run experiments yet. you'll revisit this in Section 5).

3. Concept Extraction (22 Points)

This is the first agentic step: you will use the LLM itself to decompose questions into prerequisite knowledge.

3.1 Pilot Sample Selection (3 pts)

From your `enriched_questions.csv` (Section 1.4), select 8 questions spanning at least 3 different `subject_area` values and at least 3 different difficulty levels (your 8 don't need to

cover every combination, but shouldn't all cluster at one difficulty level — this matters for later analysis). Document your selection (question IDs, text, gold answers, distractors, subject area, difficulty) in a table. This pilot set is used to design and validate your extraction prompt before scaling to the full 100.

3.2 Concept Extraction Prompting (8 pts)

1. Design a prompt (with structured output, building on Section 1.3) that asks the LLM to extract exactly 3 prerequisite concepts required to answer a given question, ordered from hardest/most advanced (concept 1) to easiest/most basic (concept 3).
 - A "concept" should be a generalizable piece of knowledge or skill (e.g., "thermal conductivity varies by material"), not a fact specific to this question's exact wording, and should not directly restate or give away the answer choice.
2. Define a Pydantic schema for this output, e.g., concepts: List[ConceptItem] where each ConceptItem has name, description, and difficulty_rank.
3. Few-shot prompting: you may include 1-2 worked examples directly in your prompt (i.e., a question + its ideal concept decomposition, written by you) to steer the model toward the granularity and style you want. This is optional but often improves consistency. if you use it, show your few-shot example(s) and briefly note whether it changed the outputs compared to a zero-shot version.
4. Run this extraction for all 8 pilot questions and save results to concepts_pilot.csv (question_id, concept_1, concept_2, concept_3, with descriptions).

3.3 Calibration / Sanity Check (6 pts)

1. Pick 2 of your 8 pilot questions (ideally at different difficulty levels) and manually write out what you (as the human annotator) would consider the "ideal" 3-concept decomposition, with brief justification for the hardest→easiest ordering.
2. Compare your manual decomposition to the model's output for these 2 questions:
 - Is the hardest→easiest ordering sensible, or did it sometimes invert difficulty?
 - Are the 3 concepts meaningfully distinct, or do any overlap/restate each other at different granularity?

3. If you spot systematic issues (redundant concepts, gives away the answer, bad ordering) across your pilot questions, revise your prompt and re-run on the pilot before proceeding. Document what you changed and why.

3.4 Scaling to the Full Dataset (5 pts)

1. Using your validated prompt from 3.2/3.3, run concept extraction on all 100 questions from enriched_questions.csv.
2. Save results to concepts_all.csv (question_id, concept_1, concept_2, concept_3, with descriptions).
3. Spot-check 5 additional random rows (outside your original pilot) and report whether the same quality holds at scale, or whether you notice new failure patterns that didn't appear in your 8-question pilot.

4. Generating Simulated Naive-Human Answers (28 Points)

4.1 Prompt Design: Two Strategies (6 pts)

For each question and each knowledge level $k = 1, 2, 3$ (meaning the simulated person does not know concepts 1 through k , using your hardest→easiest ordering), design prompts implementing two strategies:

- Strategy A. Persona Suppression: "You are a person who has never learned about [concept list]. Answer the following question as best you can."
- Strategy B. Reasoning Trace: "Simulate the step-by-step reasoning of a person who knows everything relevant to this topic except [concept list], then give their final answer."

Show one fully worked example prompt (any question, any k) for each strategy.

4.2 Pilot Generation (8 pts)

1. Using your 8 pilot questions from Section 3, for all combinations of $k=1,2,3$ and both strategies ($8 \times 3 \times 2 = 48$ generations), call the LLM using your structured output parser from Section 1.3 (reuse the answer/reasoning/confidence schema).
2. Save results to simulated_responses_pilot.csv with columns: question_id, k , strategy, simulated_answer, reasoning, confidence, raw_output.

3. Display a few example rows, including at least one where the simulated answer differs from the gold answer.

4.3 Expected Naive Answers: Pilot (7 pts)

For each (question, k) pair in your pilot set, you need a reference point representing what a real naive person might plausibly answer.

1. For the 2 questions you manually annotated in 3.3, write out for each $k=1,2,3$ what you'd expect a naive person to answer, and which distractor (if any) that corresponds to. Justify your reasoning.
2. For the remaining 6 pilot questions, use the LLM (structured output) to propose a "missing concept $k \rightarrow$ likely distractor" mapping with justification. Review all 6 and either accept or correct each. document any corrections and why.
3. Save to `expected_naive_answers_pilot.csv` (question_id, k, expected_answer, justification, source: manual/llm-accepted/llm-corrected).

4.4 Strategy Selection & Scaling (7 pts)

1. Based on your pilot results (4.2-4.3), pick one strategy (A or B) to use going forward. Justify your choice using concrete evidence from the pilot. e.g., agreement with expected naive answers, qualitative reasoning quality, or anything you noticed in 4.2's example outputs.
2. Select 30 additional questions (random sample) from the remaining 92 in `enriched_questions.csv`, spanning a range of subject_area and difficulty values.
3. For these 30 questions $\times k=1,2,3$ using your chosen strategy only (90 generations), repeat the generation process from 4.2. Save to `simulated_responses_scaled.csv`.
4. Generate expected naive answers for these 30 questions using the LLM-assisted approach from 4.3.2 (no manual annotation required at this scale), with a spot-check on 5 random rows, report your agreement rate. Save to `expected_naive_answers_scaled.csv`.

5. Failure Analysis (16 Points)

5.1 Strategy Comparison (Pilot Only) (5 pts)

Using your pilot results (8 questions $\times$ 3 k $\times$ 2 strategies), plot, for each strategy, the % of simulated answers matching the gold answer and the % matching the expected naive answer, as a function of k . Does this pattern support the strategy choice you made in 4.4? If the pilot data is ambiguous or points the other way, say so honestly. you don't need to redo Section 4, just reflect on what you'd do differently.

5.2 Knowledge Leakage (Combined Dataset) (6 pts)

"Leakage" is when the model's reasoning still relies on a concept it was told the simulated person doesn't know. e.g., if concept 1 is "thermal conductivity varies by material" and the $k=1$ reasoning still argues from that fact, the model's own knowledge has leaked into a persona that's supposed to lack it. Define a concrete criterion for detecting this in the reasoning text, then measure the leakage rate as a function of k . Does it increase or decrease as more concepts become "forbidden"? Break this down further by difficulty and subject_area: do harder questions or certain subjects leak more, and why might that be? Show 3 annotated examples with the leaked concept highlighted in the reasoning text.

5.3 Failure Mode Classification (Combined Dataset) (5 pts)

For every (question, k) pair, classify the outcome as one of: matches expected naive (success), matches gold despite missing concepts (over-suppression / leakage-driven), or matches neither (off-distribution). Plot a stacked bar chart of these three categories across $k=1,2,3$, and again broken down by difficulty. Is "matches neither" more common at high or low difficulty, and what might explain that? Finally, pick 2 "matches neither" cases and hypothesize what went wrong: bad concept extraction, an ambiguous expected-answer mapping, or model confusion.

6. Comparative Summary (5 Points)

6.1 Summary Table (2 pts)

Using your pilot data (8 questions, both strategies): Rows = Strategy A and Strategy B, each at $k=1,2,3$. Columns: % match gold, % match expected naive, % leakage, % off-distribution.

6.2 Confidence Calibration (2 pts)

Using your combined dataset (chosen strategy): plot the model's reported confidence against whether the simulated answer matched the expected naive answer. Is the model's confidence calibrated to this task (predicting a naive person), or does it seem to just reflect the model's confidence in the true answer regardless of the persona? Give evidence for your conclusion.

6.3 Best Configuration (1 pts)

Based on 6.1 (pilot), which (strategy, k) combination produced the most "human-like naive" outputs, and does this match the strategy you chose to scale with in 4.4? If your combined-dataset results (Section 5) tell a different story at scale, note that too.

Final Discussion

Think through these. you don't need to submit written answers, but be prepared to discuss.

1. Is "pretend you don't know X " fundamentally different from "predict what a naive person would say"? Which is the model actually doing, based on your leakage analysis?
2. If concept extraction (Section 3) had been done by a different LLM, or by a human expert, how much do you think your downstream results would change? Where is your pipeline most sensitive to errors in early steps?
3. How would this task change if you had access to real survey data from intro-level students instead of heuristic "expected naive" answers?
4. What are the risks of using LLM-simulated "naive learner" data to train educational tools (e.g., adaptive tutors)? Where might simulated data mislead more than it helps?

Task 4 – Image Segmentation Using Clustering Methods

Image segmentation is one of the most classic and extensively studied tasks in computer vision, with a wide range of applications—from segmenting tumors in medical images to identifying and separating different elements of a road scene in autonomous driving, such as lanes, vehicles, pedestrians, and traffic signs. Despite this, image segmentation tasks often face substantial challenges related to data. One of the primary issues is the need for high-quality, pixel-level annotations, which are both time-consuming and expensive to produce—especially in domains like medical imaging, where expert knowledge is required. To address these challenges, unsupervised methods have been deployed to automatically generate segmentation masks required for training these data. One such method is clustering, which groups pixels based on their features—such as color, intensity, or spatial information—into different segments. Although this might be a complex problem in cluttered images it is possible for some image datasets. In this section you are going to create segmentation masks for players from the [football player segmentation](#) dataset.

0. Understanding Segmentation

The goal of image segmentation is to divide an image into meaningful parts or regions. This is done by assigning a label to each pixel. Note that this is different from classification where each image is assigned to one or multiple labels. This makes segmentation essential for tasks requiring precise localization, such as identifying individual players in a football game or isolating specific objects in complex scenes.

Segmentation tasks can be categorized into multiple categories. The simplest one is **semantic segmentation**, which assigns each pixel a class label based on the object or region it belongs to, without distinguishing between different instances of the same class. For example, in a football game image, all pixels belonging to players would be labeled as "player," regardless of which specific player they represent. **Instance Segmentation** is another type of segmentation whose goal goes beyond semantic segmentation by not only labeling pixels by their class but also differentiating between individual instances of the same class. In the football player dataset, instance segmentation would assign unique labels to each

player, enabling separation of player 1 from player 2, even though both belong to the "player" class. Finally **Panoptic Segmentation** combines semantic and instance segmentation, labeling every pixel with both a class and, for countable objects, an instance ID. In the context of the football dataset, panoptic segmentation would assign unique IDs to each player while labeling background elements like the ground or crowd as single, separated regions. The focus of this project is mainly on semantic segmentation. You are going to use simple clustering approaches for this assignment. The reason is that it is possible to cluster image pixels where each cluster becomes one segment of the image. With this knowledge you are ready to start the next parts. Good luck!

1. Dataset Loading

Load the dataset from the Kaggle link. Because this section contains no training and not the whole dataset is needed. Sample 50 images from the dataset and use them for the rest of this section. This dataset contains two directories. The annotations which contain a json file of all segmentation masks and the images. The images are in size of 1920*1080. Down scale images to 1/8 or 1/16 of the original size or otherwise you will run into performance and memory issues in the upcoming parts.

2. Creating Features (20 points)

Feature creation is the most important part of the clustering. Pixel features should be crafted in such a way that meaningful regions which include players get allocated into one or multiple clusters. Like the example in the course lectures, it is possible to do the clustering only using pixel colors. But note that this simple feature will not likely yield good results. You should try different more complex feature selection methods and report the results for each of them. For example, one method can be concatenating the pixel colors with the pixel positions in the image. Report and evaluate all feature selection methods you explored.

3. Cluster pixels (30 points)

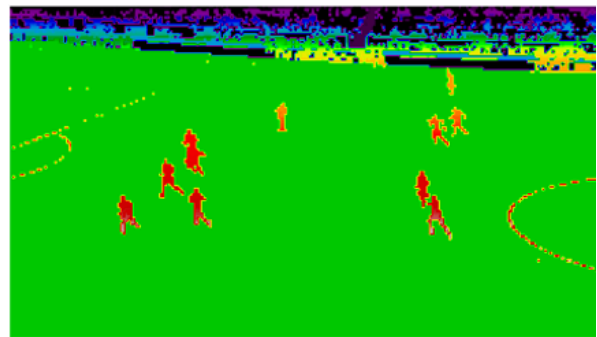
Many different clustering methods from simplest ones like K-Means to more advanced approaches such as density-based clustering (e.g., DBSCAN) and hierarchical methods (e.g.,

Agglomerative clustering). Use these methods or any similar clustering approach that you find more suitable. The most important task of this part is tuning the clustering parameters. For example if you use K-Means, the selection of K can be challenging especially in this task where it might be difficult to exactly determine the K based on prior knowledge. Use metrics like Silhouette score, inertia, and any suitable metric you find. Output of this part should be multiple clusters. It is not necessary that each player is clustered in only one cluster. You can visualise your clusters to see the results of your segmentation. Note that you might see that each player is segmented into multiple clusters which is fine up to this part. But be careful that there might exist a problem if for example parts of the ground and a player parts are in the same cluster. An example of the clustering can be like this:

Original Image



Clustering Results



4. Filtering and Merging (15 points)

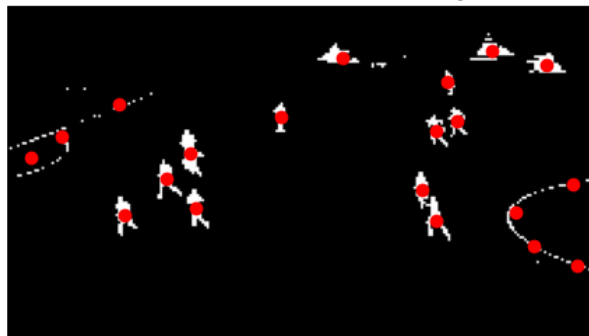
More than the problem mentioned in the last part, there is the problem of segments that do not contain the players, for example the ground. To filter these segments you can set different thresholds on the size of each cluster and apply filters that can drop unrelated clusters. After that you need to merge smaller clusters that are positionally next to each other in the image to create bigger clusters.

After this part you should notice each player is segmented into one or two clusters. An example is shown below.



5. Even More Clustering! (15 points)

In this step, you will generate a binary mask from the clusters obtained in previous parts, where each pixel is assigned a value of 1 (True) for regions corresponding to players and 0 (False) for non-player regions, such as the ground or other background elements. The binary mask should isolate players, ensuring that only their pixels are marked as 1, while all other areas (e.g., the field, crowd, or other objects) are marked as 0. After creating the mask, apply clustering again to identify connected components within the mask and compute their centroids. Finally, visualize the results to display the binary mask and the identified components with their centroids. An example of such a binary mask is shown below. White color corresponds to value 1.



6. More Advanced Features! (10 points bonus)

Use a pretrained convolutional model like ResNet or HRNet to create rich feature representations from the input image. Note that these models decrease the height and width

of the model at each step. Down scaling the input image can result in losing details of the image and unacceptable results. I suggest that after generating the features apply DBSCAN and try to tune the parameters so that DBSCAN assigns -1 labels to players pixels.

Explain why DBSCAN shows this behaviour in this part. Finally repeat parts 3 and 4 to get you final player segments.

7. Evaluation (20 points)

To evaluate your segmentations, use the ground-truth segments provided in the JSON file located in the annotation folder of the football player segmentation dataset. For evaluation, create a binary mask from your clustering results (from part 5), where pixels corresponding to players are assigned a value of 1, and all other pixels (e.g., ground, crowd) are assigned 0. Similarly, load the ground-truth annotations from the JSON file and convert them into binary masks in the same format, with player pixels set to 1 and non-player pixels set to 0. The goal is to measure the quality of your segmentation by comparing your predicted binary masks against the ground-truth masks. What we want to measure is the amount of overlap between the output segments and ground truth labels. In this part, we will use the Dice coefficient and Intersection over Union (IoU) as evaluation metrics. Compute and report these metrics on a sample of 50 images from the dataset.

Dice Coefficient:

The Dice coefficient is used to measure the similarity between two sets. In the context of image segmentation, it measures the overlap between the predicted segmentation and the ground truth. It is calculated as twice the area of intersection divided by the sum of the predicted and ground truth areas. A Dice score of 1 indicates perfect agreement, while a score of 0 indicates no overlap.

Intersection over Union (IoU):

Intersection over Union (IoU), is a common evaluation metric for segmentation tasks. It is defined as the area of overlap between the predicted and ground truth masks divided by the area of their union. Unlike the Dice coefficient, IoU gives more penalty to mismatches and is

often considered a stricter metric. A higher IoU value shows better segmentation performance, with 1 representing a perfect match and 0 indicating no overlap.

$$IoU = \frac{|A \cap B|}{|A \cup B|}$$

Questions

- Provide an example of a segmentation task where each type of segmentation—semantic, instance, and panoptic—is appropriate and sufficient, and explain why each type suits the specific task.
- Explain the difference between the Dice coefficient and Intersection over Union (IoU) as evaluation metrics in image segmentation. In what scenarios might one be preferred over the other?
- Imagine a scenario where we need to cluster a large set of images into multiple categories. Methods such as K-means can become computationally expensive and inefficient when applied directly to high-dimensional image data. One approach to address this issue is to use an autoencoder architecture to reduce the dimensionality of the images before clustering. Explain how autoencoders can be used for this purpose. How does this method help improve the efficiency and effectiveness of clustering algorithms like K-means on large image datasets?

Notes

- Upload your work as a zip file in this format on the website: DS_CA5_[Std number].zip. If the project is done in a group, include all of the group members' student numbers in the name.
- Only one member must upload the work if the project is done in a group.
- We will run your code during the project delivery, so make sure your results are reproducible¹.
- Written answers to the questions are **not required**. These questions are for your own preparation; we will discuss these concepts and your design choices during the project delivery.

¹ Your code will be executed as-is in a fresh environment (e.g., a clean Docker container or VM) to verify that it runs without errors, and your outputs (e.g. visualizations and dataframes) are same (or effectively the same) as the result you've uploaded. Use practices like randomizing with seeds, to make the result more reproducible, but we also understand that due to the design of the project, reproducing exact same output on every section is not possible.